MODULUS

Retail Operations Platform

Playwright E2E Test Plan

Version 1.0

1. Overview

This document defines the Playwright end-to-end test suite for the Modulus Retail Operations Platform. The suite tests the fully composed application — all four micro-frontends loaded together — rather than testing each application in isolation. Isolation testing is the responsibility of the Vitest unit test suites within each application.

The test suite covers one complete, realistic operations workflow. The workflow is designed to cross application boundaries: it creates data in the Inventory MFE, updates data in the Orders MFE, and verifies the result in the Analytics MFE. This cross-MFE coverage is what distinguishes E2E testing from unit testing in a micro-frontend context.

All tests run against the production shell URL. They do not run against a local development server. This is deliberate: the suite validates the deployed system, not a local approximation of it.

2. Test Infrastructure

2.1 Configuration

The playwright.config.ts file is located at the monorepo root under playwright/. Key settings:

```
import { defineConfig } from "@playwright/test";

export default defineConfig({
  testDir: "./tests",
  fullyParallel: false,        // Sequential: tests share state
  retries: 1,                  // One retry on CI to reduce flake
  reporter: [
    ["html", { outputFolder: "playwright-report", open: "never" }],
    ["github"],                // Annotates PRs with failure details
  ],
  use: {
    baseURL: process.env.PLAYWRIGHT_BASE_URL,
    screenshot: "only-on-failure",
    video: "retain-on-failure",
    trace: "retain-on-failure",
  },
  projects: [
    { name: "chromium", use: { ...devices["Desktop Chrome"] } },
  ],
});
```

2.2 Page Object Models

All selector logic is encapsulated in page object models. Test files contain no raw selectors. This makes the suite maintainable: if a selector changes, it is updated in one place.

Page Object	File	Responsibilities
LoginPage	pages/LoginPage.ts	Navigate to login, fill credentials, submit, assert redirect
ShellPage	pages/ShellPage.ts	Navigate sidebar sections, assert active section, assert user info
InventoryPage	pages/ InventoryPage.ts	Search, filter, add product, edit product, assert table rows and badges
OrdersPage	pages/OrdersPage.ts	Filter by status, open detail, update status, assert timeline
AnalyticsPage	pages/AnalyticsPage.ts	Assert metric cards, assert charts rendered, change date range

2.3 Test Data

The test suite uses a fixed set of test data identifiers that are guaranteed to exist in the MSW seed. These identifiers are defined in a single constants file at tests/constants.ts and imported by all test files. Tests do not hardcode IDs or names inline.

```
// tests/constants.ts
export const TEST_PRODUCT = {
  name: "E2E Test Product",
  sku: "E2E-001",
  category: "Electronics",
  price: "29.99",
  stock: "100",
};

export const TEST_ORDER_ID = "o-seed-001"; // pending order in seed
export const TEST_CREDENTIALS = {
  email: "ops@modulus.com",
  password: "password123",
};
```

3. User Journey Test Suite

The suite consists of a single test file: tests/operations-workflow.spec.ts. The file contains one describe block with 14 sequential test steps. Steps are sequential because each step depends on the state produced by the previous step.

The describe block uses a shared page fixture initialised in beforeAll. The fixture logs in once and holds the authenticated session for all 14 steps. A beforeEach hook is not used because login state must persist across steps.

Step	MFE	Action	Assertion	Key Selector
1	Shell	Navigate to shell URL	Login page renders with email and password fields visible	`[data-testid="login-form"]`
2	Shell	Submit valid ops credentials	Shell layout renders: sidebar visible, user name displayed in nav	`[data-testid="shell-sidebar"]`
3	Inventory	Click Inventory in sidebar	Inventory MFE loads: product table renders with at least one row	`[data-testid="product-table"]`
4	Inventory	Click Add Product, fill form with TEST_PRODUCT values, submit	Success toast appears; new product row appears in table matching name and SKU	`[data-testid="product-row-E2E-001"]`
5	Inventory	Search for TEST_PRODUCT.sku in search input	Table shows exactly one row; stock badge shows green (100 units)	`[data-testid="stock-badge-green"]`
6	Inventory	Click Edit on TEST_PRODUCT row, change stock to 3, save	Success toast appears; stock badge on the row changes to red	`[data-testid="stock-badge-red"]`
7	Orders	Click Orders in sidebar	Orders MFE loads: order table renders with rows	`[data-testid="order-table"]`
8	Orders	Filter by status "pending"	Table shows only rows with status badge "pending"	`[data-testid="status-badge-pending"]`
9	Orders	Click on TEST_ORDER_ID row to open detail view	Order detail renders: customer info, line items table, and timeline visible	`[data-testid="order-detail"]`
10	Orders	Select "processing" from status dropdown, confirm in modal	Status badge in detail view updates to "processing"; timeline gains a new entry	`[data-testid="status-badge-processing"]`
11	Orders	Navigate back to order table, filter by "pending"	TEST_ORDER_ID row is no longer visible in the pending	`[data-testid="order-table"]`

Step	MFE	Action	Assertion	Key Selector
			filter	
12	Analytics	Click Analytics in sidebar	Analytics MFE loads: all four metric cards and all four charts render	`[data-testid="analytics-dashboard"]`
13	Analytics	Assert revenue chart has rendered data points	LineChart contains at least one data point element; chart is not in empty state	`[data-testid="revenue-chart"] .recharts-dot`
14	Shell	Click logout in top navigation	Shell redirects to login page; sidebar is no longer visible	`[data-testid="login-form"]`

4. Failure Handling

4.1 Failure Blocks Deployment

The Playwright job runs as a stage in the shell CI/CD pipeline after the deploy stage. If any test step fails, the Playwright process exits with a non-zero code. GitHub Actions interprets this as a job failure, which marks the overall pipeline run as failed.

The deployment itself has already completed before the Playwright stage runs. A Playwright failure does not roll back the deployment. It signals that the deployed version did not pass acceptance testing and requires investigation before the pipeline status is considered green.

4.2 Report Publishing

The HTML report is published to GitHub Pages after every pipeline run, regardless of whether tests passed or failed. The if: always() condition on the publish step ensures the report is available even when tests fail — which is precisely when engineers need it most.

Reports are published to a path that includes the GitHub Actions run ID, ensuring each run produces a distinct URL. The format is:

```
https://<org>.github.io/modulus/e2e-reports/<run_id>/index.html
```

The Slack alert message includes this URL as a direct link, so the engineer receiving the alert can reach the report in one click.

4.3 Slack Alert Content

The Slack alert fires only on failure (if: failure() condition). It does not fire on a passing run. The alert message must contain:

- The text "Modulus E2E Failure" as the message header
- The name of the failed test step
- The MFE the failed step was operating in
- The GitHub Actions run ID
- A direct URL to the GitHub Pages report for this run
- A link to the failed pipeline run in GitHub Actions

The Slack channel receiving alerts must be the same channel monitored by the engineering team. The webhook URL is stored in the SLACK_WEBHOOK_URL repository secret.

4.4 Retry Behaviour

The Playwright config sets retries: 1 on CI. A failing test is retried once before being marked as failed. This reduces false failures from transient network issues during remote loading but does not mask genuine application bugs.

A test that fails on the first attempt and passes on the retry is reported as "flaky" in the HTML report. Flaky tests are tracked and investigated. A test that is consistently flaky without a network cause is a defect in the test, not an accepted state.

5. Selector Strategy

All selectors use data-testid attributes. CSS class selectors, XPath, and text-based selectors are not used in page object models. This isolates the test suite from styling changes and from content changes in the MSW seed.

Every interactive element and every element that serves as an assertion target must have a data-testid attribute added during implementation. The naming convention is:

```
[data-testid="<component>-<element>"]

Examples:
  [data-testid="login-form"]
  [data-testid="product-table"]
  [data-testid="product-row-E2E-001"]   // row with SKU suffix
  [data-testid="stock-badge-green"]
```

```
[data-testid="order-detail"]
[data-testid="status-badge-processing"]
[data-testid="analytics-dashboard"]
[data-testid="revenue-chart"]
```

data-testid attributes are added in application code during the implementation phase. They are not stripped in production builds: their presence in production code is intentional and supports future monitoring and automation.

6. Out of Scope

- Cross-browser testing: the suite runs in Chromium only for the portfolio demonstration. Expanding to Firefox and WebKit is documented as a future enhancement but is not implemented.
- Visual regression testing: pixel-level screenshot comparison is not implemented. Visual correctness is validated by human review during the Phase 9 polish walkthrough.
- Load or performance testing: the suite does not test under concurrent user load. Performance is measured via Lighthouse in Phase 9.
- API contract testing: MSW handler correctness is covered by unit tests in packages/types. E2E tests do not test the mock layer directly.
- Accessibility automation: automated accessibility assertions are not included in the E2E suite. Accessibility is validated via Lighthouse in Phase 9.